

1. ¿Qué ventajas tiene escribir el código SQL únicamente en la capa del modelo?

Una ventaja es que se mantiene una mejor organización del código, ya que toda la lógica de acceso a la base de datos está en un solo lugar. Esto hace que sea más fácil de mantener y modificar, porque si cambia una consulta, solo se cambia en el modelo y no en varias partes del sistema.

También ayuda a separar responsabilidades, ya que el controlador solo se encarga de manejar la lógica de la aplicación y no de escribir consultas SQL.

2. ¿Qué es SQL injection y cómo se puede prevenir?

SQL injection es un tipo de ataque en el que un usuario introduce código SQL malicioso en una entrada, con el objetivo de manipular la base de datos.

Se puede prevenir utilizando consultas parametrizadas, donde los valores se envían por separado y no se concatenan directamente en el string del query, evitando que se ejecute código malicioso.

3. ¿Por qué en la práctica las credenciales de Supabase viven en un archivo .env y no directamente en database.js?

Se guardan en un archivo .env para proteger la información sensible, como usuarios y contraseñas.

Si se escribieran directamente en el código y se subieran a GitHub, podrían pasar cosas como:

- * Que cualquier persona pueda ver las credenciales y conectarse a la base de datos sin permiso.
- * Que alguien modifique o borre información de la base de datos, afectando el sistema.

4. ¿Qué problema resuelve usar un pool de conexiones?

El pool de conexiones evita tener que abrir y cerrar una conexión cada vez que se hace una petición.

Esto mejora el rendimiento del sistema, porque reutiliza conexiones ya existentes y reduce el tiempo de espera, además de evitar sobrecargar el servidor de base de datos.

5. Ejemplo de query inseguro vs parametrizado

Query inseguro:

```
const sql = "SELECT * FROM games WHERE title = '" + titulo + "'";
```

Query seguro:

```
const sql = "SELECT * FROM games WHERE title = $1";  
pool.query(sql, [titulo]);
```

En el query inseguro, si el usuario escribe algo como ' OR 1=1 --, se modifica la consulta y puede devolver todos los registros.

En cambio, en el query parametrizado, ese valor se trata como texto y no como código SQL, por lo que no se ejecuta la inyección.